

Rapport de Projet Final : SmartFarm AI

Auteur : Zakaria Ech-chafi

Module : Applications Distribuées & Big Data

1. Contexte et problématique

La gestion des exploitations agricoles modernes nécessite une supervision constante des paramètres environnementaux. L'absence de centralisation des données capteurs et le manque d'outils d'aide à la décision limitent souvent la productivité.

Problématique : Comment concevoir une solution IoT distribuée, capable de collecter des données en temps réel, de les traiter via une architecture moderne et d'apporter une assistance intelligente (IA) pour optimiser les rendements ?

2. Architecture générale

Le projet repose sur une architecture distribuée en trois couches :

Couche Ingestion (IoT) : Simulation de capteurs envoyant des données (Température, Humidité, CO2) via WebSocket.

Couche Traitement (Backend) : API FastAPI traitant les flux, orchestrant la logique métier et interrogeant le modèle Gemini pour l'analyse prédictive.

Couche Présentation (Frontend) : Interface Dashboard React.js consommant les données en temps réel via WebSockets.

3. Composants implémentés

main.py (FastAPI) : Point d'entrée de l'API. Gère les routes du dashboard, le chat avec AgriBot AI et l'orchestration des services.

sensor_ws.py : Script de simulation qui génère les flux de données IoT et maintient la connexion persistante avec le serveur.

AgriBot AI : Module intégrant l'API Gemini pour interpréter les données capteurs et répondre aux questions de l'utilisateur sur l'état de la serre.

4. Technologies et bibliothèques utilisées

FastAPI : Pour un backend performant et asynchrone (gestion native des WebSockets).

React.js & Tailwind CSS : Pour une interface utilisateur réactive et un design professionnel.

Recharts : Bibliothèque utilisée pour la visualisation dynamique des données (graphiques AreaChart).

Google Gemini API : Moteur d'intelligence artificielle utilisé pour l'analyse sémantique et les recommandations agricoles.

Docker : Pour la conteneurisation et l'isolation des dépendances.

5. Intégration Docker

L'intégration Docker permet de garantir que l'environnement d'exécution est identique sur toute machine.

Dockerfile : Définit l'image de base (Python 3.10-slim), installe les dépendances via `requirements.txt` et expose le port 8000.

docker-compose.yml : Simplifie le déploiement en orchestrant les services. Il permet de lancer l'application en une seule commande (`docker-compose up`), assurant ainsi la portabilité totale du projet.

6. Résultats obtenus, limites et perspectives

Résultats

Le dashboard affiche en temps réel les données de multiples zones (Tomates, Fraises, etc.).

La communication WebSocket est stable et réactive.

L'assistant AgriBot AI fournit des analyses pertinentes basées sur les conditions de température et d'humidité en cours.

Limites

Dépendance réseau : L'IA nécessite une connexion internet constante pour interroger les serveurs de Google (Gemini).

Quotas API : Le système peut être limité par les quotas de requêtes de l'API gratuite en cas de sollicitation massive.

Perspectives

Persistence : Ajouter une base de données (ex: InfluxDB) pour stocker l'historique sur le long terme.

Automation : Étendre les capacités de l'IA pour déclencher automatiquement des actions correctives (ex: activation automatique des ventilateurs).

Scalabilité : Utiliser un orchestrateur (type Kubernetes) pour gérer davantage de zones agricoles si le projet s'agrandit.